

NAME:S.ROSHINI

DEPARTMENT:B.TECH INFORMATION TECHNOLOGY

CLASS: IT-C (2nd YEAR)

REGISTER NO:922525205148

Mini Web Application - EXPENSE TRACKER

Technologies Used:

HTML, CSS, JavaScript, Django, Django REST

Framework,SQLite

Abstract / Project Overview

Expense Tracker is a full-stack web application developed to help users manage their daily expenses in a simple and organized way. The application allows users to add, view, edit, and delete expense records. The frontend is developed using HTML, CSS, and JavaScript, while the backend is developed using Django and Django REST Framework. SQLite is used as the database for storing expense information. The application also calculates the total amount spent and displays the number of expenses. REST APIs are used to communicate between the frontend and backend. The application was tested using Postman to verify all CRUD operations.

Problem Statement

Managing daily expenses manually can be difficult because users may forget their spending details or find it difficult to calculate their total expenditure.

Traditional methods such as notebooks or simple notes do not provide an organized way to manage expense records. Therefore, a simple web application is required to store expense information and allow users to easily add, view, update, and delete their expenses

Objectives

- To develop a simple expense management application.
- To allow users to add expense details.
- To display stored expenses.
- To provide edit and delete functionality.
- To calculate total spending.
- To display the number of expenses.
- To implement REST APIs.
- To store expense data in a database.
- To create a responsive user interface.
- To test the application using Postman.

Proposed Solution

The proposed Expense Tracker provides a centralized interface for managing expense records. Users can enter the expense title, amount, category, date, and description. The data is sent to the Django REST API and stored in the SQLite database. Users can view all saved expenses and perform update or delete operations. The application automatically calculates the total spending and expense count.

Features

1. Add Expense

Users can enter:

- Expense title
- Amount
- Category

- Date
- Description

2. View Expenses

All stored expenses are displayed on the application.

3. Edit Expense

Users can modify existing expense information.

4. Delete Expense

Users can remove unwanted expense records.

5. Total Spent

The application calculates the total amount spent.

6. Expense Count

The application displays the total number of expense records.

7. Responsive UI

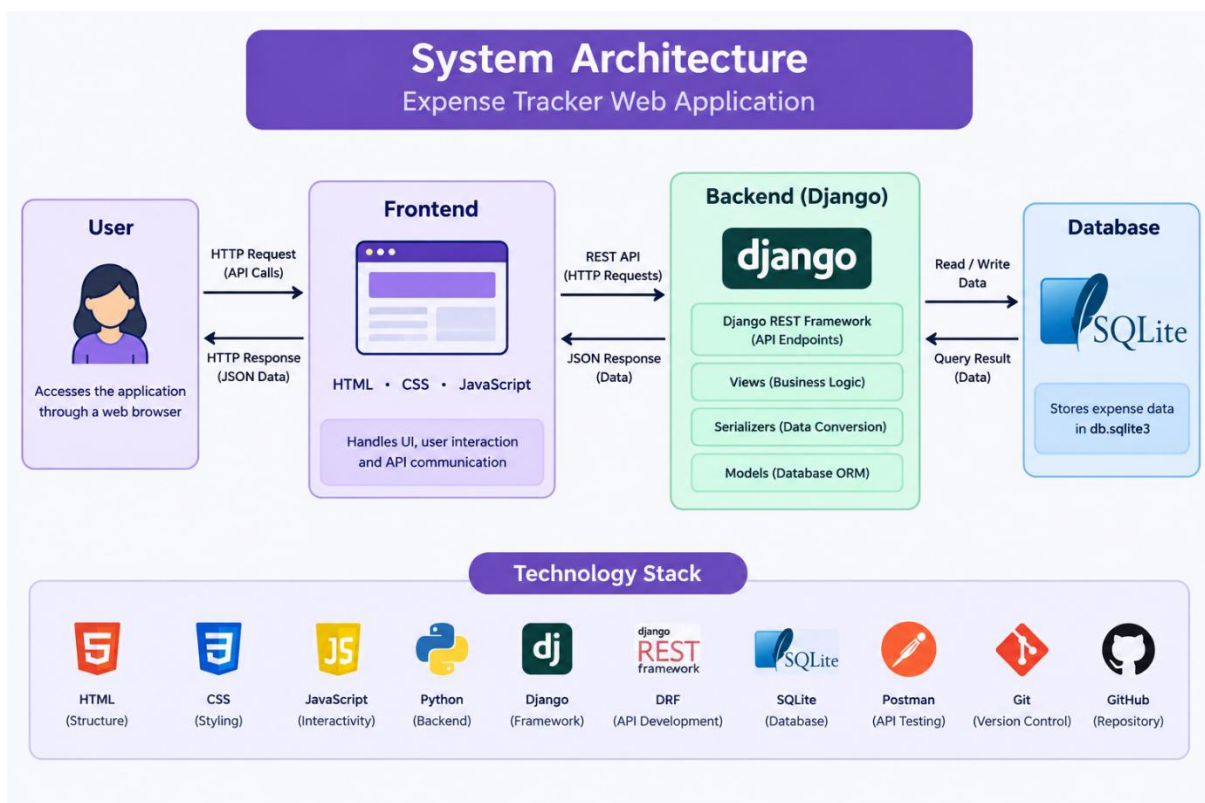
The interface works on desktop and mobile screen sizes.

Technology Stack

Technology	Purpose
HTML	Structure of frontend
CSS	Styling and responsive design
JavaScript	Frontend functionality and API communication
Python	Backend programming

Django	Backend framework
Django REST Framework	REST API development
SQLite	Database
Postman	API testing
Git	Version control
GitHub	Source code repository
VS Code	Development environment

System Architecture



Database Design / ER Diagram

The main database entity is:

Expense

Fields:

Expense

id

title

amount

category

date

description

Frontend Implementation

index.html

Responsible for:

- Expense form
- Expense list
- Total Spent
- Expense Count
- Buttons

style.css

Responsible for:

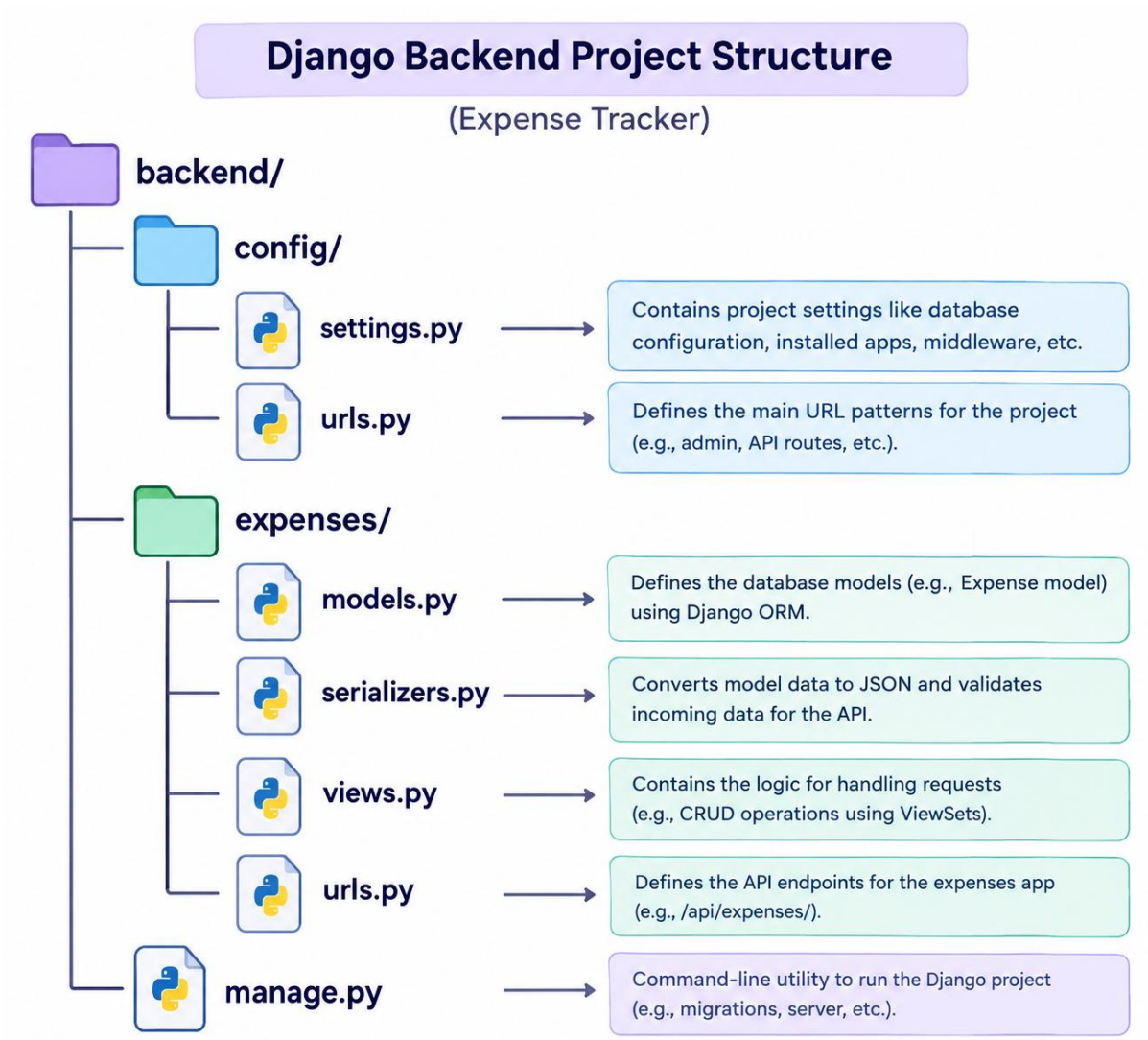
- Purple/violet theme
- Cards
- Buttons
- Layout
- Responsive design
- Animations

script.js

Responsible for:

- Sending POST requests
- Getting expense data
- Editing expenses
- Deleting expenses
- Calculating total
- Calculating expense count

Backend Implementation



REST API Documentation

Operation	HTTP Method	Endpoint	Expected Response
Create	POST	/api/expenses/	201 Created
Read	GET	/api/expenses/	200 OK
Read One	GET	/api/expenses/{id}/	200 OK
Update	PUT	/api/expenses/{id}/	200 OK
Delete	DELETE	/api/expenses/{id}/	204 No Content

POST Example:

```
{  
  "title": "Lunch",  
  "amount": 150,  
  "category": "Food",  
  "date": "2026-09-14",  
  "description": "Lunch at college"  
}
```

PUT Example:

```
{  
  "title": "Postman Updated",  
  "amount": 300,  
  "category": "Shopping",  
  "date": "2026-09-14",  
  "description": "Updated using Postman"  
}
```

CRUD Operations

CREATE

The user enters expense information and submits the form.

Frontend sends:

`POST /api/expenses/`

The backend stores the data in SQLite.

READ

The frontend requests:

`GET /api/expenses/`

The backend returns the stored expenses.

UPDATE

The user selects Edit and changes the expense details.

`PUT /api/expenses/{id}/`

DELETE

The user selects Delete.

`DELETE /api/expenses/{id}/`

The selected record is removed.

Testing

Test Case	Method	Expected Result	Status
Get expenses	GET	Expenses displayed	✓ Pass
Add expense	POST	New expense created	✓ Pass
Update expense	PUT	Expense updated	✓ Pass
Delete expense	DELETE	Expense deleted	✓ Pass
Mobile view	UI	Page fits screen	✓ Pass

And mention your actual Postman results:

GET → 200 OK

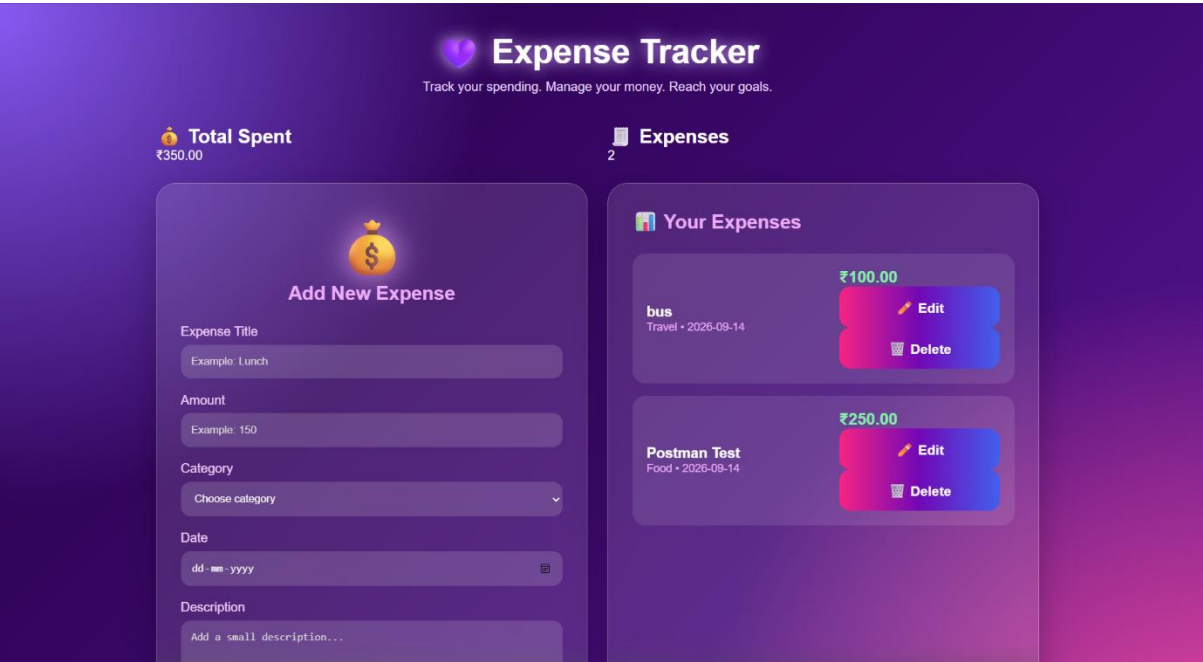
POST → 201 Created

PUT → 200 OK

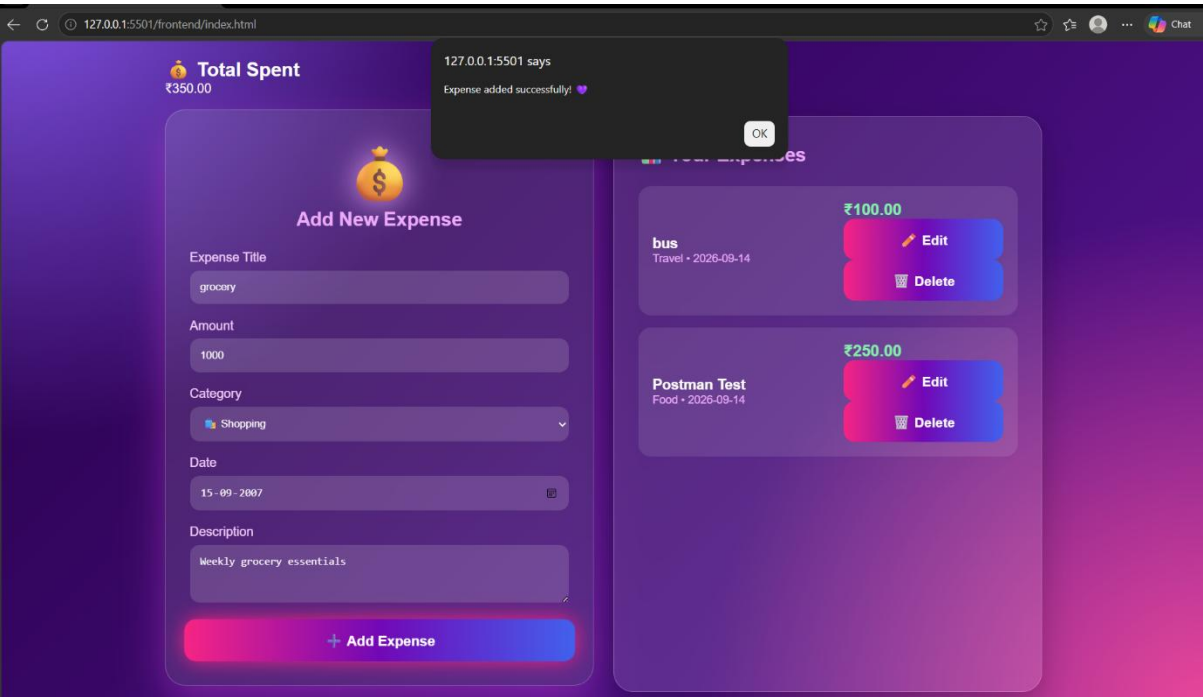
DELETE → 204 No Content

Screenshots:

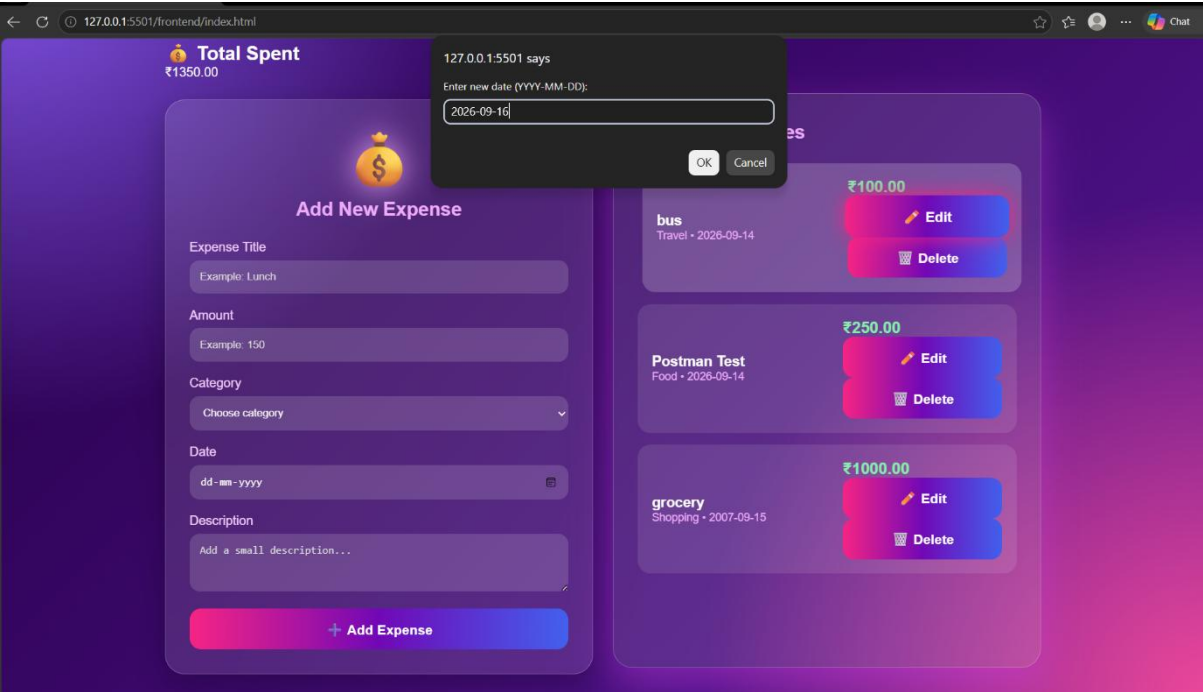
Homepage:



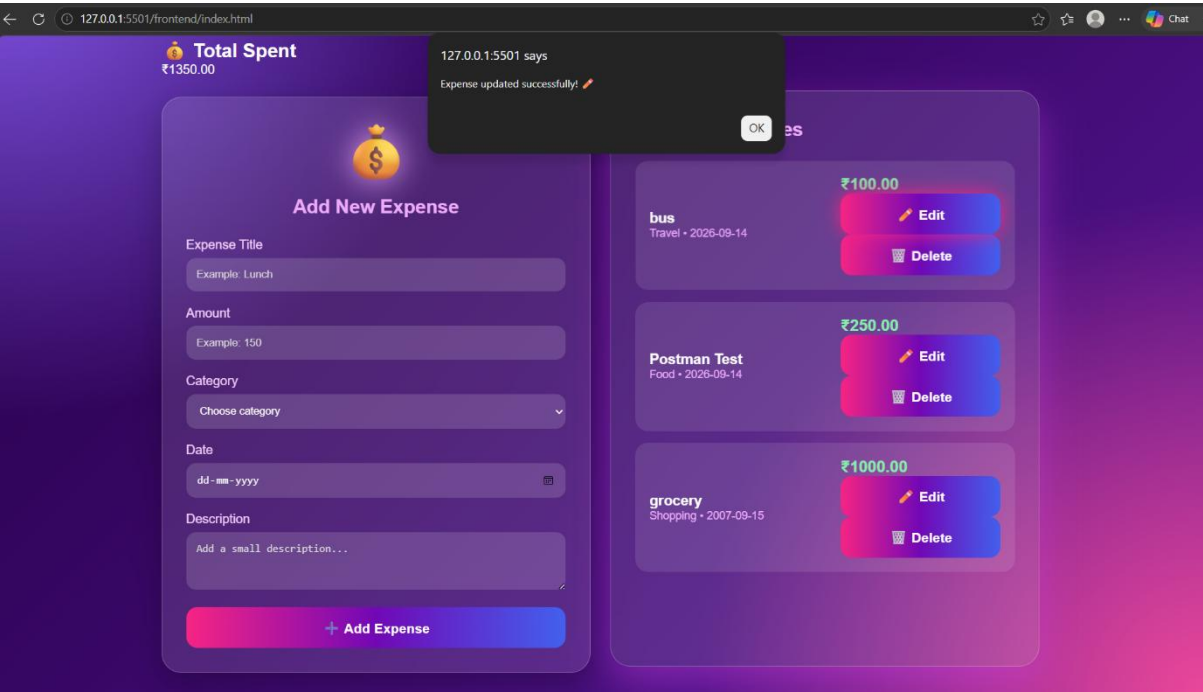
Expense added:



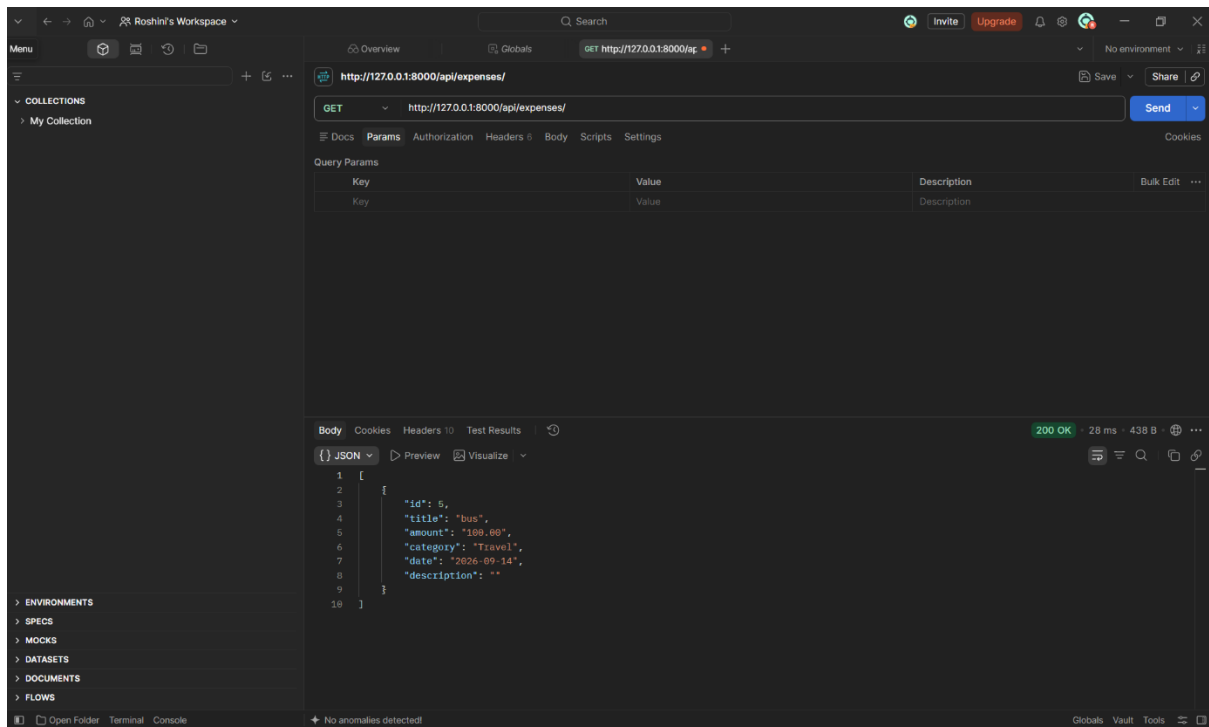
Edit:



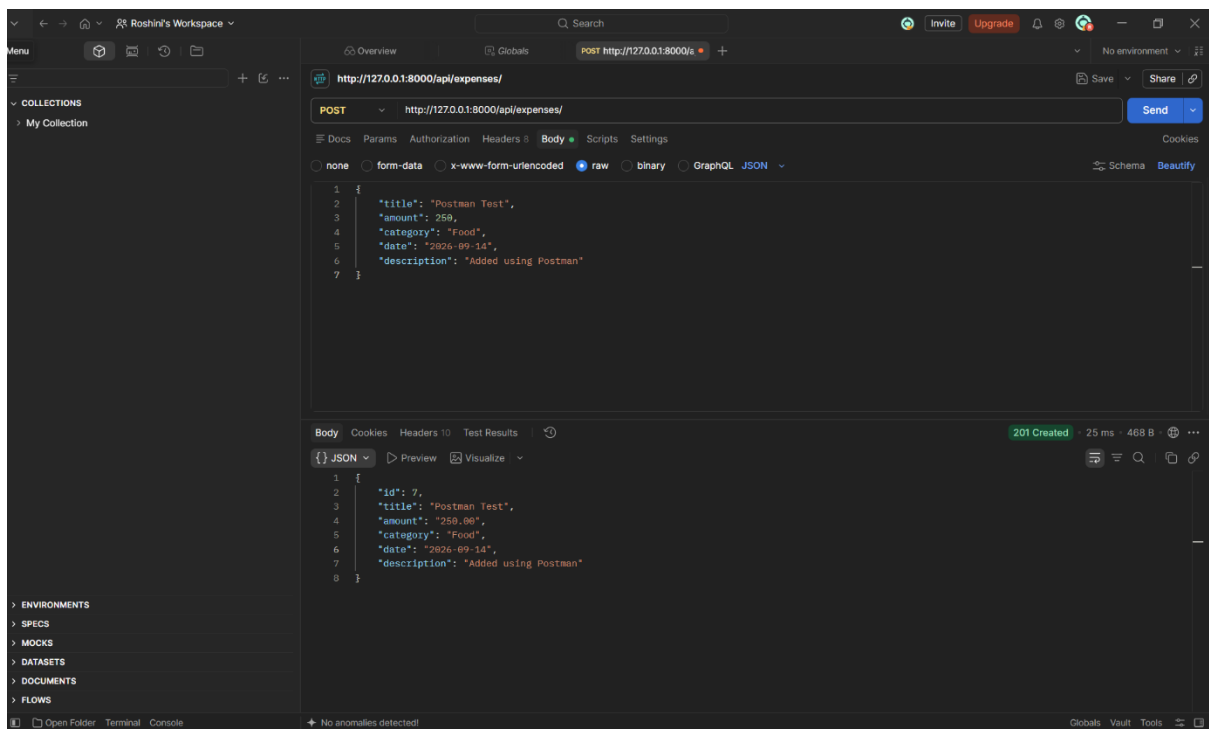
Update:



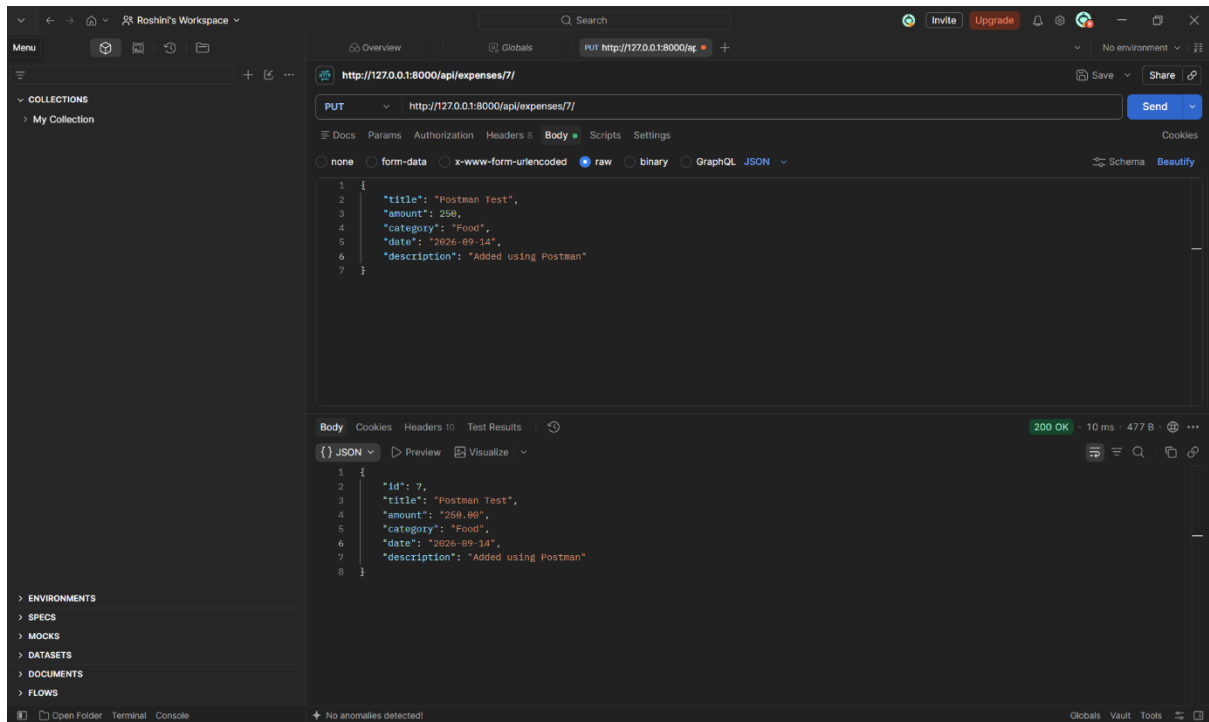
Postman GET



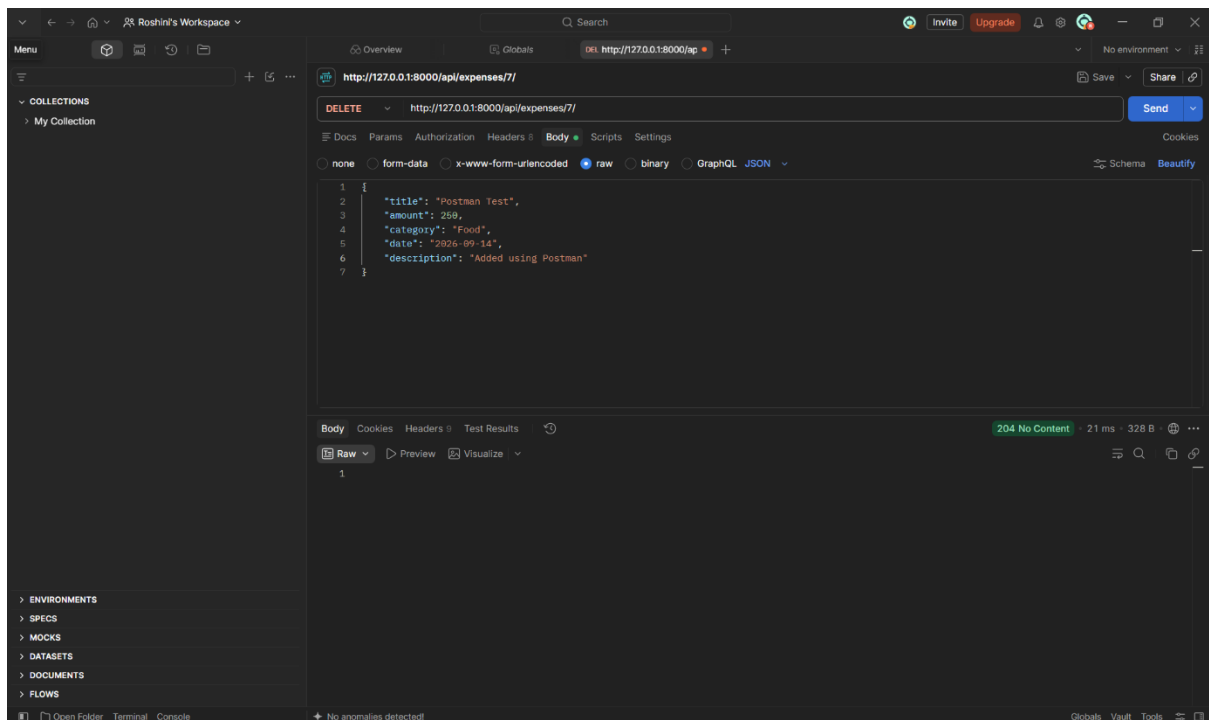
Postman POST



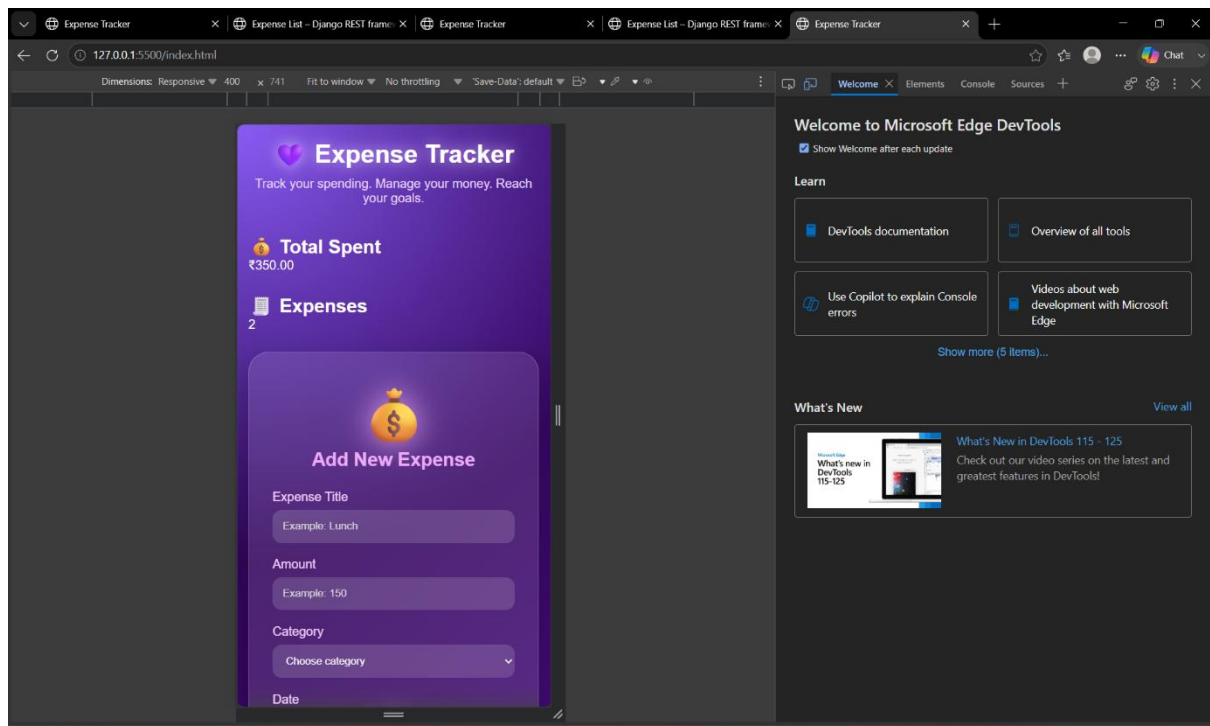
Postman PUT:



Postman DELETE



Mobile view



Responsive Design

Mobile testing:

Example:

The application was tested using browser device emulation to verify its responsive behavior. The layout adapts to smaller screen sizes, and the form and expense cards fit within the mobile screen without horizontal overflow.

Conclusion

The Expense Tracker successfully demonstrates the development of a full-stack web application using HTML, CSS, JavaScript, Django, Django REST Framework, and SQLite. The application supports complete CRUD operations and provides a simple responsive interface for managing expenses.

REST APIs were tested using Postman, and the project was maintained using Git and GitHub. The project helped demonstrate practical knowledge of frontend development, backend API development, database management, testing, and version control.